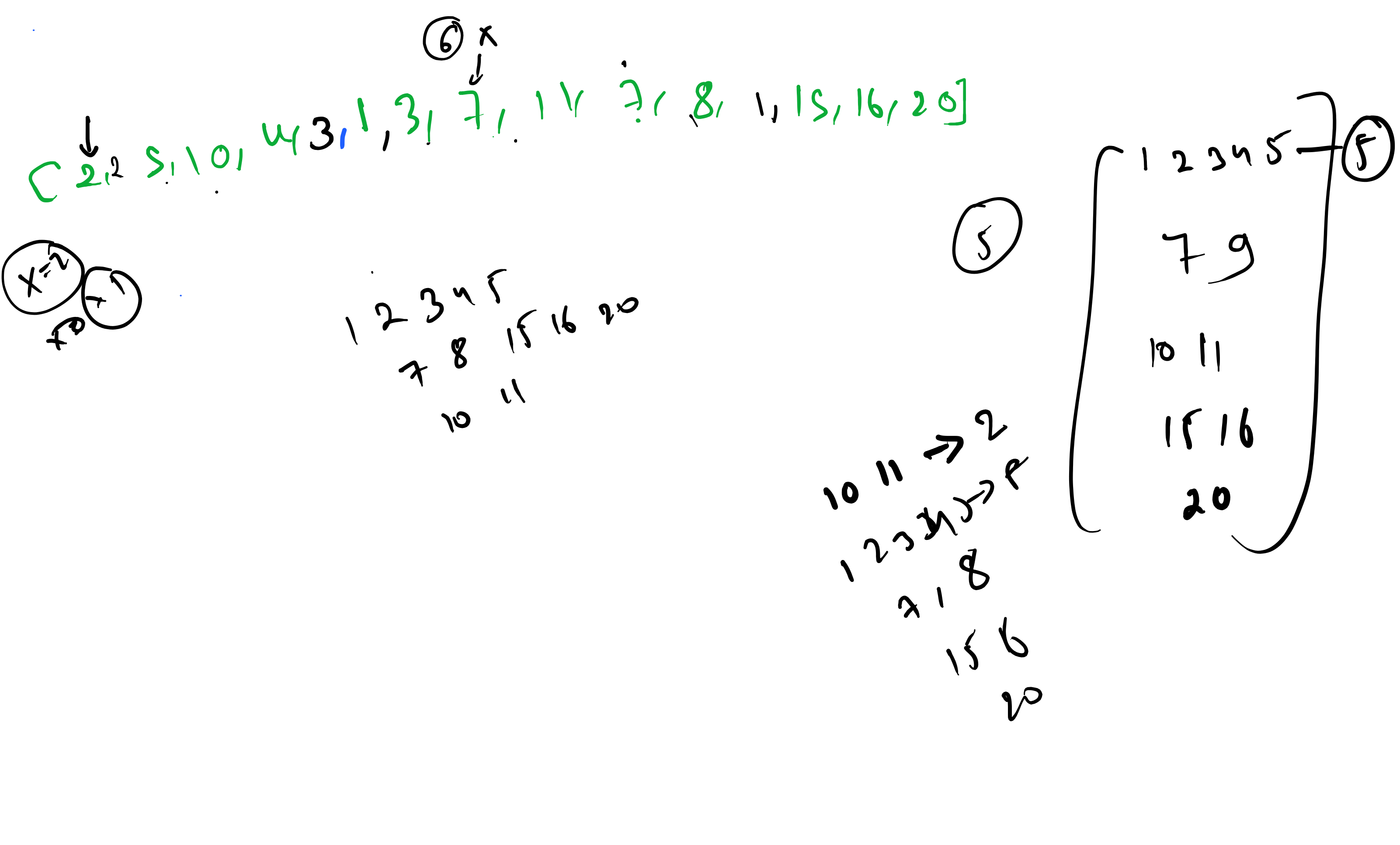
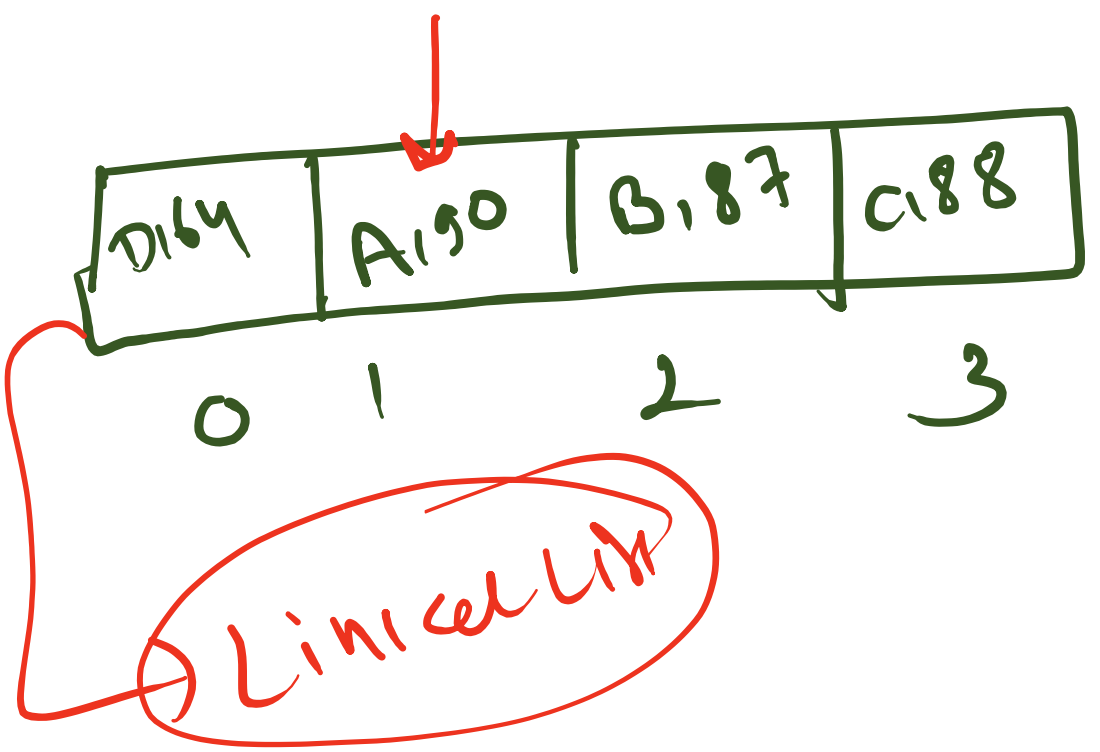
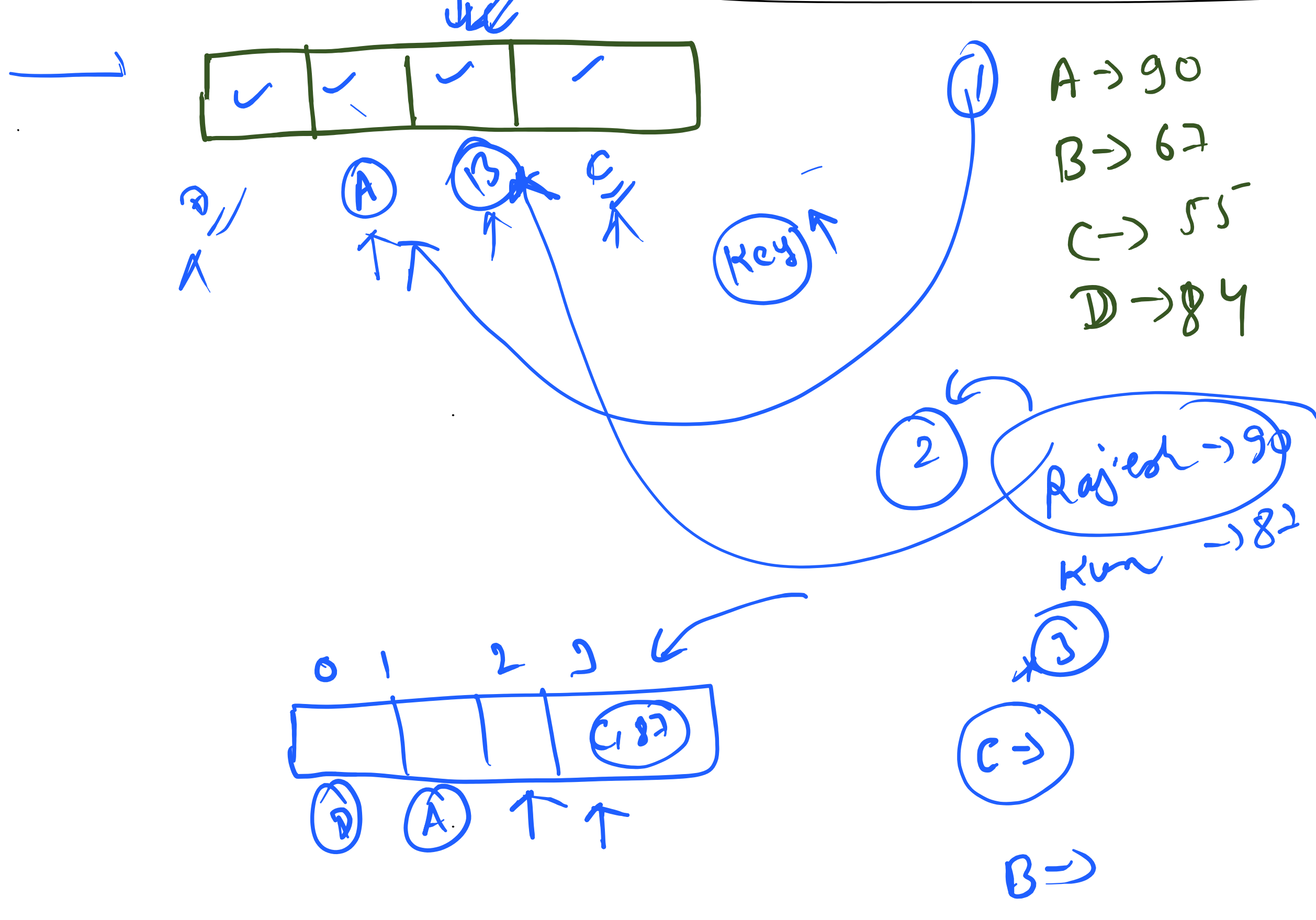
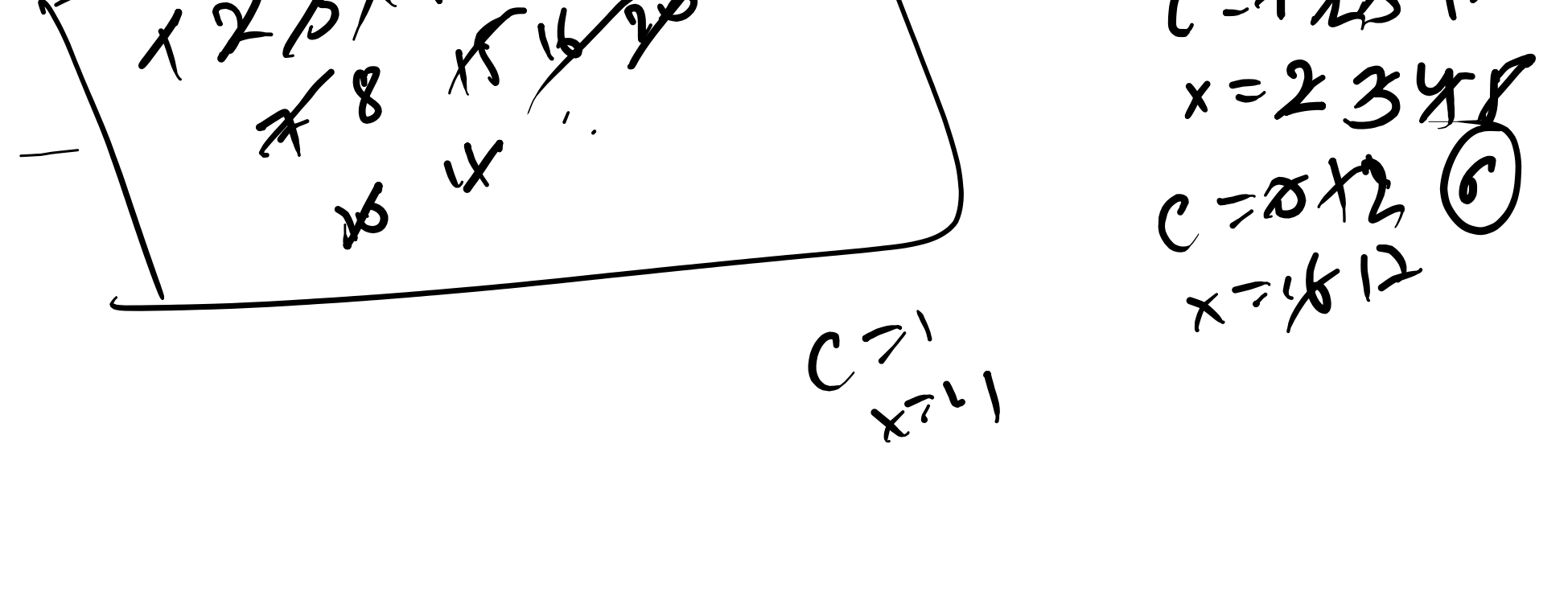


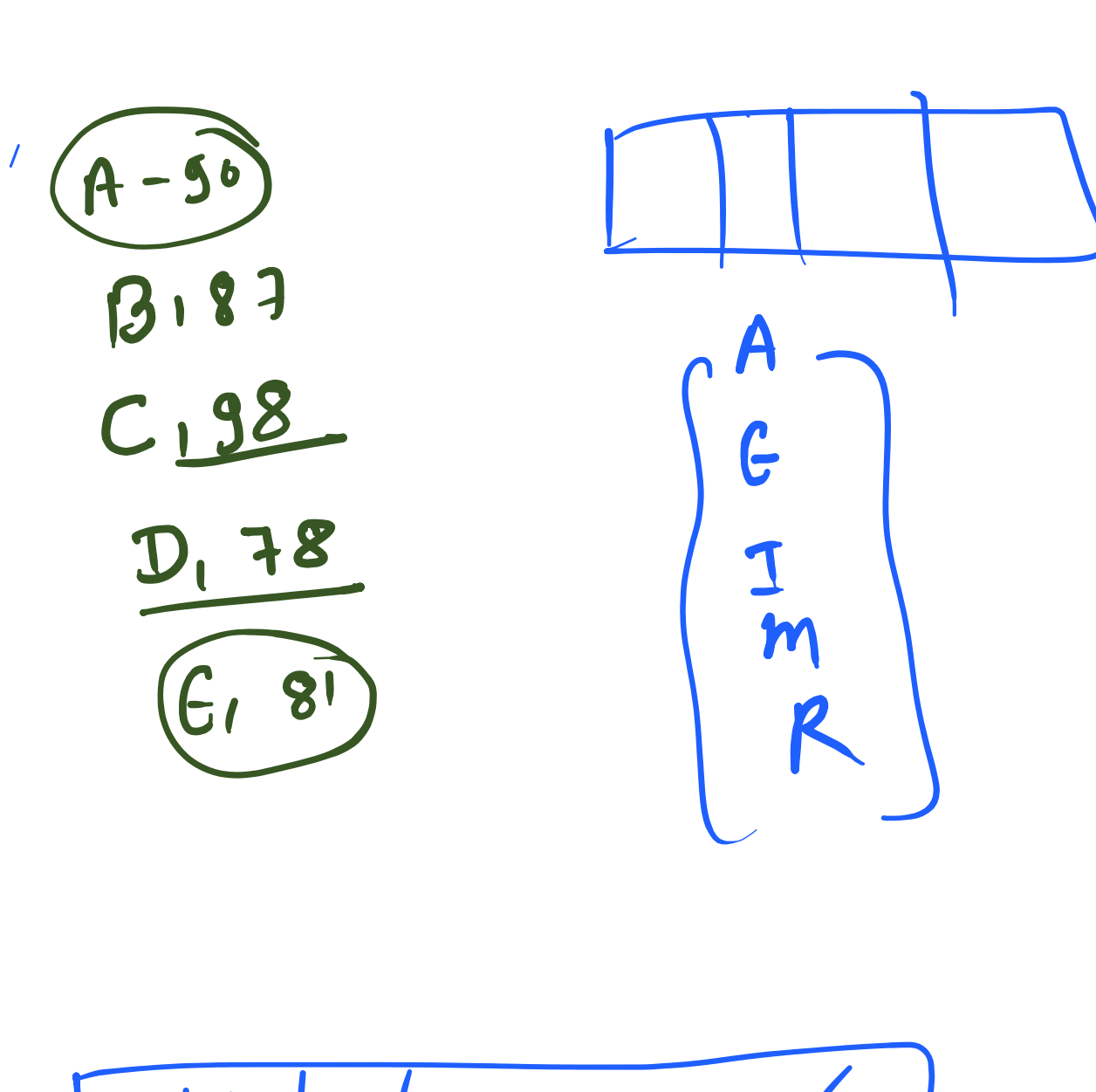
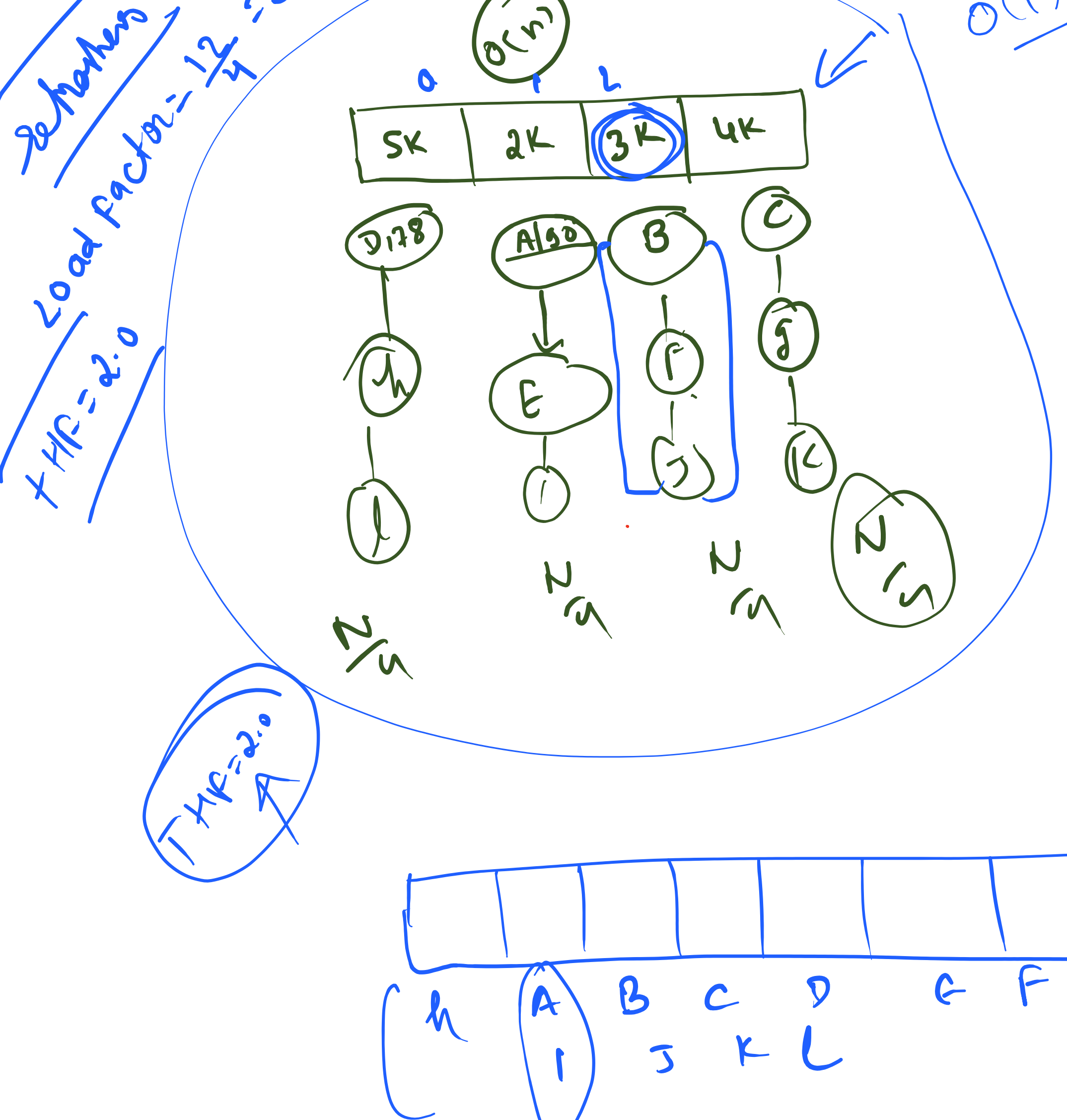


```
public static int LongestSequence(int[] arr) {
    HashSet<Integer> set = new HashSet<>();
    for (int i = 0; i < arr.length; i++) {
        set.add(arr[i]);
    }
    int ans = 0;
    for (int i = 0; i < arr.length; i++) {
        int x = arr[i];
        if (set.contains(x) && !set.contains(x-1)) {
            int c = 0;
            while (set.contains(x)) {
                set.remove(x);
                c++;
                x++;
            }
            ans = Math.max(ans, c);
        }
    }
    return ans;
}
```



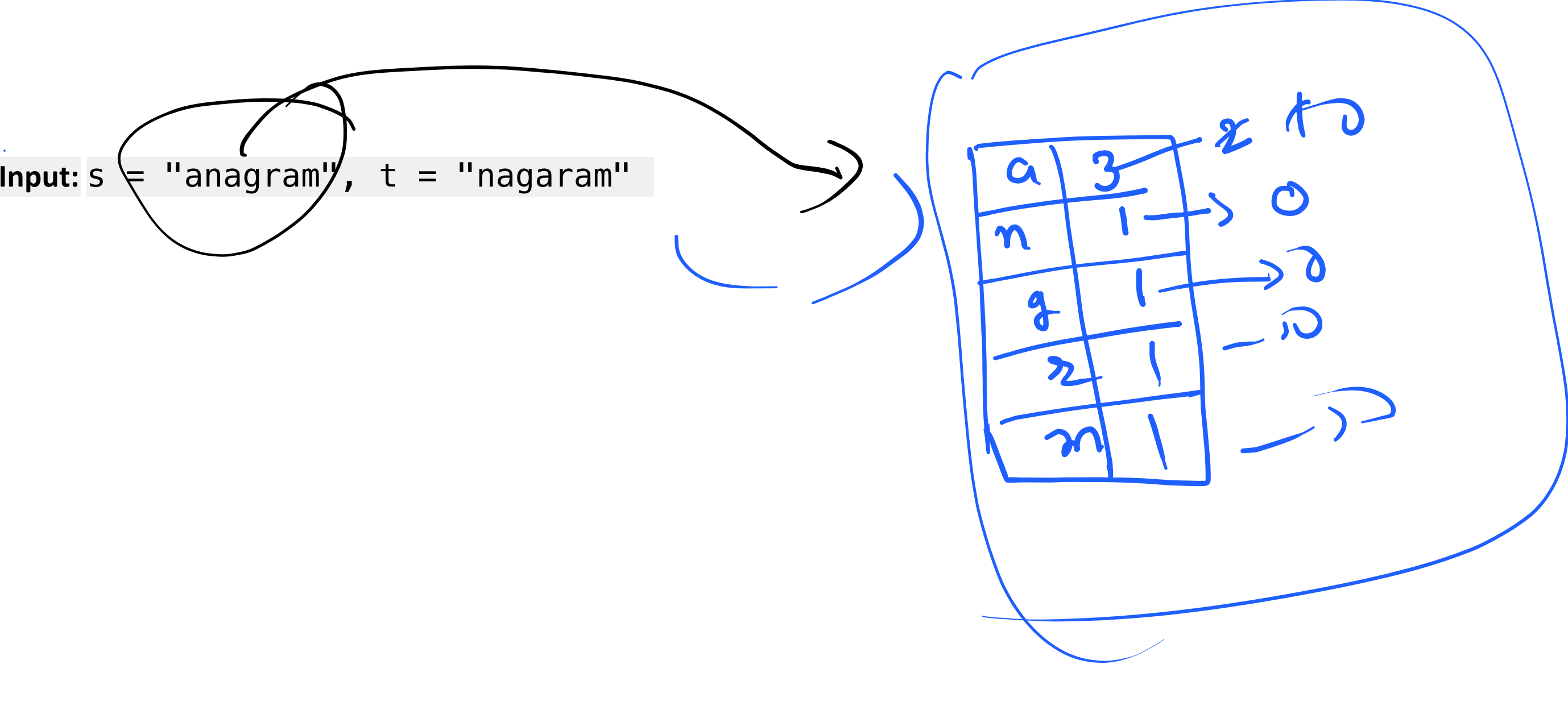
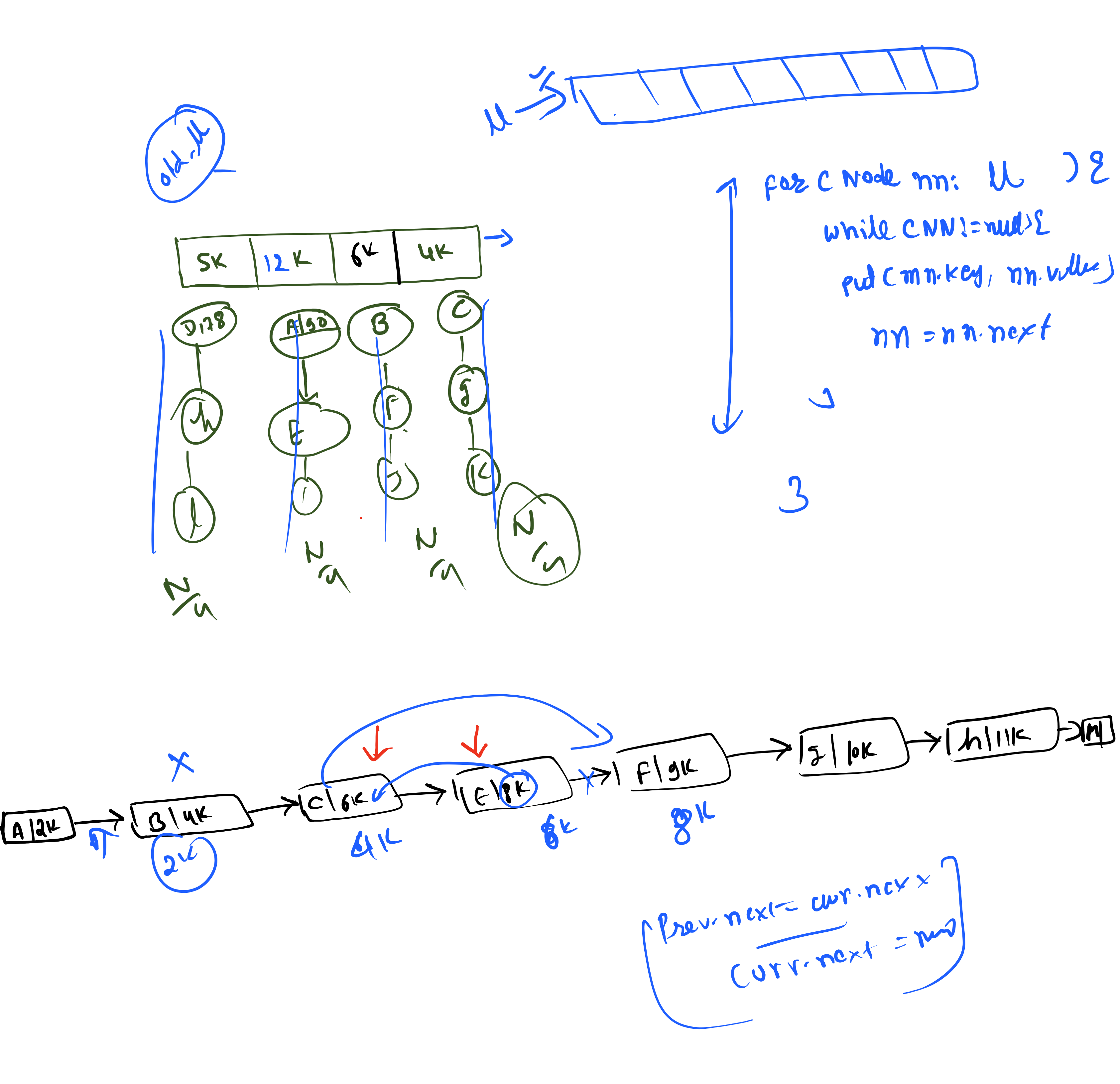
hashFunc → ASCII * 4

A → 90	65 * 4 = 1
B → 87	66 * 4 = 2
C → 88	67 * 4 = 3
D → 64	68 * 4 = 0
E → 98	69 * 4 = 1



class Node {
key
value
next
}

hash(null)
hash(new)



```
String[] arr = {"eat", "tea", "tan", "ate", "nat", "bat"};

public static List<List<String>> GroupAnagrams(String[] arr) {
    HashMap<String, List<String>> map = new HashMap<>();
    for (int i = 0; i < arr.length; i++) {
        String s = arr[i];
        String key = Gen_Key(s);
        if (!map.containsKey(key)) {
            map.put(key, new ArrayList<>());
        }
        map.get(key).add(s);
    }
    return map.values();
}
```

